Project 2: TCP Layer in Petnet

Overview

You will build an implementation of TCP for the Petnet TCP/IP stack. It needs to conform to the Petnet API, and to the reduced TCP standard described here. Note that Petnet also provides utility code, helper functions, and generic containers that you may choose to use in your implementation. You can extra credit by implementing additional parts of the TCP standard.

Host Network Configuration:

Petnet uses a tap-based network interface to directly transmit and receive packets from user space. Therefore, you will need to configure a tap interface and a bridge to connect the tap device to the system's network stack.

Note that these instructions assume you will be using a localhost-only configuration (where Petnet can only accept connections from the local host, not remote machines). You can find instructions online to extend the configuration to support remote connections if you like.

Please note that this configuration is quite hack-y and there is probably a better way to do this (so please don't copy and paste it and say we taught you this was right! It's just a simple way to get something that works for our purposes in this project).

Enabling Petnet: We provide a script `petnet_up.sh` that will set up the bridge and tap device. The contents of that script are:

```
sudo ip link add name petnet_bridge type bridge
sudo ip tuntap add dev petnet_tap1 mode tap user cs1652
sudo ip link set petnet_tap1 master petnet_bridge
sudo ip addr add 192.168.201.1/24 brd + dev petnet_bridge
sudo ip link set dev petnet_bridge up
sudo ip link set dev petnet_tap1 up
sudo ip link set petnet_bridge address a6:94:0d:b7:92:e9
```

This script should work in most cases, but adjustments may be needed if it conflicts with your local system environment. In particular, you may need to check the following:

1. If another local network's subnet overlaps with 192.168.201.0/24 then you should change the subnet on line 7 to a non-overlapping range. You can use whatever subnet you want here, but it would be best to stick with a private non-routable subnet.
2. If you are extremely unlucky and have another local device with a MAC address of a6:94:0d:b7:92:e9, then you will need to change the MAC address on line 10 to some other arbitrary value.
3. If your username is not cs1652 (the default username in the course VM image) you should replace cs1652 in line 5 with the username you will be using to test your project.

If you're not sure exactly what we mean in points 1&2 don't worry about it – it's unlikely you have a configuration that conflicts, and we'll talk more about IP addresses and MAC addresses in a few weeks.

To enable Petnet, run the script (it will prompt you for your password, which is cs1652 in the course VM):

```
$ ./petnet_up.sh
```

Note that if you restart your VM, you will need to re-run the script.

If the environment was configured correctly, you should see the petnet bridge and tap device as part of the active network interfaces. If you run the command `ip a`, you should see the following contained in the output (minus the line wrapping).

```
3: petnet_bridge: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc noqueue
state DOWN group default qlen 1000
    link/ether a6:94:0d:b7:92:e9 brd ff:ff:ff:ff:ff:ff
    inet 192.168.201.1/24 brd 192.168.201.255 scope global petnet_bridge
       valid_lft forever preferred_lft forever
4: petnet_tap1: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 qdisc fq_codel
master petnet_bridge state DOWN group default qlen 1000
    link/ether 9e:85:94:0c:44:57 brd ff:ff:ff:ff:ff:ff
```

To test your project, you need to have Petnet enabled. But, if you would ever like to disable Petnet, you can run the provided script `petnet_down.sh`

Petnet Configuration:

Once the network environment is configured you will need to set an IP address for the Petnet stack. By default, Petnet is set to use 192.168.201.12, which falls into the subnet we specified as part of the petnet tap/bridge configuration. If you changed the Petnet subnet in the config file above, you will need to change this address to be inside the range of the subnet you chose. To change the Petnet IP address you will need to edit `config.json` in the top level of the petnet directory, and update the "ipv4_address" value.

You should also make sure that the "net_device" value matches the tap device you created as well (petnet_tap1). When this is set you will be able to open network connections to the host using the IP address set on the bridge (192.168.201.1 by default).

Building Petnet:

To compile Petnet, you need to install one dependency (this should be preinstalled on the course VM):

```
$ sudo apt install libunwind-dev
```

Once the environment is setup, you can compile Petnet by running make from the top-level directory of the Petnet project:

```
$ make
```

Petnet Overview

Petnet is a library that provides a user level networking stack that executes in the background of an application. This background execution is handled by a set of helper threads that are created by the `petnet_run()` function that is invoked by a petnet application during initialization.

Every petnet application will include at least 3 separate threads of execution: (1) a main application thread that will be used by petnet to handle all socket API calls, (2) a packet receive thread that will asynchronously handle all incoming packets from the network, and (3) a timeout thread that will be used to handle all timeout events. This means that by working inside petnet you will necessarily be working inside a multi-threaded environment, and will need to explicitly handle the concurrency issues that arise from it. However, to keep things tractable, Petnet provides abstractions that simplify the concurrency problems; as long as you follow a few simple rules you should be able to ignore most of the thread-based complexity. We will explain those rules later in this document.

Applications interact with Petnet through an API that overlaps with a subset of the Linux socket API. The petnet API is for the most part functionally equivalent to the standard socket calls, but is differentiated by the addition of a '`petnet_`' prefix. So, for example, the petnet version of `accept()` is `petnet_accept()`. The API should contain all the functionality necessary to implement the client and server from project 1. A small set of example applications are included in the `apps/` directory. If you want to add a new app for testing/debugging you will need to place the .c file into the `apps/` directory and add it to the 'execs' list in `apps/Makefile`.

The Petnet library implementation is included in the top level directory `libpetnet/`. Libpetnet includes the core user space network stack, as well as a set of network helper functions and container implementations organized into a set of subdirectories.

- `core/` : This contains the core network stack implementation organized by protocol. This is where all of the code you write for project 2 will live. In particular you will be editing `tcp.c` and `tcp_connection.h`. Also worth noting are the files `udp.c` and `udp_endpoint.c` which include a functional UDP layer to demonstrate how to interact with the IPv4 and socket APIs.

- `util/` : This contains a set of helper functions for dealing with common networking data types. This includes an abstract IPv4 address datatype as well as a checksum implementation. In addition, `inet.h` includes host to network byte order conversion functions.

- `petlib/` : This contains a set of container types that you are free to use as well as additional system utility functions. The most likely container you will need to interact with is the ringbuffer implementation in `pet_ringbuffer.[ch]`. Wrapper functions for logging and memory management can be found in `pet_log.h` and `pet_util.h` respectively.

Running Petnet

Even without a functional TCP layer, Petnet can still provide some functionality. To verify that things are running correctly, you can start Petnet and try to ping it from the command line. If everything is running correctly, Petnet should respond to the ping. To do this, you just need to select one of the

sample applications and execute it from the command line. As an example, you could run the UDP test server from the top-level petnet directory:

```
petnet$ ./apps/udp_test_1_server 5555 test_message
```

Once it starts running you should be able to ping the Petnet IP address (192.168.201.12 by default) from another terminal and see the responses. If the pings timeout, this indicates an error in your configuration. If so, you should double check your settings and contact either the TA or professor if you are unable to resolve the issue.

You can test the UDP functionality with nc by running (in another terminal):

```
$ nc -u 192.168.201.12 5555
test_message
```

After running the above, nc will wait for input. You can type in the string `test_message`, followed by `Ctrl+D` (EOF) to send it to the UDP server. The UDP server should receive the message and indicate that the test passed (i.e. it received the expected input).

As you work through the project, we recommend you start with a "hardcoded" TCP implementation that does not rely on the socket API calls. In this stage, you can use the UDP test application to simply start up the petnet stack and run your TCP code.

Once you move on to connecting your TCP code to the socket API, we recommend you use a simple TCP application (e.g. the listen_server), and iteratively add functionality to the TCP layer until the application is able to run successfully to completion. Eventually, you should be able to connect to the TCP listen server using the **`nc`** command from another terminal on the host. Assuming the default configuration and listen_server running on port 5555, you would use the command:

```
$ nc 192.168.201.12 5555
```

`nc` will be a very useful debugging tool for this project, so if you are unfamiliar with it, please refer to the man page or search online for more information.

Implementing TCP

For this project you will only be implementing a subset of the full TCP functionality inside Petnet. Nevertheless, your version of TCP should be able to successfully communicate with any other functional TCP implementation including the one in Linux. The TCP layer you build should adhere to the following specification.

TCP Specification

The core specification for TCP is RFC 793, which you can and should fetch from https://datatracker.ietf.org/doc/html/rfc793. In general, you will implement TCP as defined in that document, except for the parts listed below.

- You only need to implement stop and wait with retransmissions

- You do not have to support outstanding connections (i.e., an incoming connection queue to support the listen backlog) in a passive open.
- You do not have to implement congestion control.
- You do not have to implement support for the URG and PSH flags, the urgent pointer, or urgent (out-of-band) data.
- You do not have to support TCP options.
- You do not have to implement a keep-alive timer
- You do not have to implement the Nagle algorithm (i.e. you do not need to buffer data to try to "fill" a segment before sending it).
- You do not have to implement delayed acknowledgements.
- You do not have to generate or handle ICMP errors.
- You may assume that simultaneous opens and closes do not occur
- You may assume that exceptional conditions such as aborts do not occur.

Chapter 3 of your textbook also serves as an excellent introduction to TCP concepts and should be read before the RFC.

Programming Considerations

The Petnet architecture is designed to provide abstractions over much of the complexity that comes from building multi-threaded applications with C. However, you need to be aware of these abstractions in order to use them effectively.

Packet Ownership

The first thing to know is that you should never have to free a packet except as part of handling an error during packet transmission. All packets are freed by the low-level device driver, either after all receive processing has completed or after a packet has been successfully transmitted into the network. The only time you should need to free a packet is if the function `ipv4_pkt_tx()` fails for some reason. In that case you need to catch the error, and free the packet you were trying to transmit. You should ensure your code is structured to ensure this invariant. The best way to do this is to encapsulate the packet send operation into a single function that creates a packet, sets the various header and data fields, and then transmits the packet immediately.

Handling Concurrency

While Petnet does make heavy use of the pthread library, it tries very hard to abstract out the complexities of the pthread API from the code needed for the project. The foundation of this approach is grounded in *reference counting* with integrated *locking*. If you have not encountered reference counting before, it is a technique for managing dynamically allocated memory objects such that deallocation is handled internally by the object itself when all existing references to the object have been dropped. A simple way to implement reference counting is by including a single integer value inside a data structure and making sure that the value is incremented or decremented when pointer references to the object are created or destroyed. This behavior is achieved by forbidding any direct copies or assignment of a pointer variable, and instead requiring that all new copies of a pointer are done using a special 'get' operation. Conversely whenever a variable containing a pointer reference is about to fall out of scope or otherwise be deleted, a special 'put' operation must be called as well. "Getting" a pointer returns the pointer value itself and increments the reference count, while "putting" a pointer value simply decrements the reference count. If the value of the reference count variable ever reaches 0, then it is

guaranteed that there are no other pointer references to the object anywhere else in the program, and so it can be safely freed. This is typically done as part of the put operation, where after decrementing the variable, a simple check is made if the new count is equal to 0. If it is, then the free operation is called immediately. If all of that sounds overwhelming, don't worry; it's really not that bad, and with a little practice makes reasoning about pointer lifetimes much easier.

In the context of Petnet, reference counting is used as part of the lifecycle management for the connection state structures. While the internals are a bit more complex than can comfortably be described here, the basic idea is that access to the connection state is managed via a well-defined API that implicitly implements reference counting for you. In fact, there is no way to access connection state other than using a reference count enabled API. The only thing you will need to worry about is making sure you drop the reference via a *put* operation whenever you are done using the pointer variable. If, after reading this, you are worried that you will create hard to debug memory leaks by forgetting to drop the reference, don't worry too much, because forgetting to drop a reference will likely cause much bigger problems for you that will be much more obvious. This is because failing to drop a reference will likely lead to a deadlock scenario due to the fact that the Petnet API integrates reference count operations with mutex locking/unlocking.

For most of the accesses to the `struct tcp_connection` structure you should be using the following API functions:

```
/*
 * Returns a locked reference to a TCP connection object corresponding to a
 * socket. The pointer behaves like a normal pointer, but you cannot free it.
 * You must unlock and release the reference before returning.
 *
 * Returns NULL on error
 */
struct tcp_connection *
get_and_lock_tcp_con_from_sock(struct tcp_con_map * map,
                               struct socket      * socket);

/*
 * Returns a locked reference to a TCP connection object corresponding to a IPV4
 * tuple. The pointer behaves like a normal pointer, but you cannot free it.
 * You must unlock and release the reference before returning.
 *
 * Returns NULL on error
 */
struct tcp_connection *
get_and_lock_tcp_con_from_ipv4(struct tcp_con_map * map,
                               struct ipv4_addr   * local_ip,
                               struct ipv4_addr   * remote_ip,
                               uint16_t             local_port,
                               uint16_t             remote_port);
```

```
/*
 * Unlocks and releases the reference to a tcp connection object after you are
 * done using it */
void
put_and_unlock_tcp_con(struct tcp_connection * con);
```

As you can see, these functions combine searching for and returning a data object with reference counter manipulation and mutex operations. *As long as you restrict yourself to these functions, most of the concurrency issues will be abstracted out of view*. The best approach is to use one of the get methods at the start of each function that serves as an entry point to your TCP implementation. These methods are already defined for you and are the stubs that are present in the tcp.c file you were given. The put method can then be called immediately before returning from those externally invoked API points.

As an example, this is my implementation of `tcp_send()`, which you are free to use:

```
int
tcp_send(struct socket * sock)
{
    struct tcp_state      * tcp_state = petnet_state->tcp_state;
    struct tcp_connection * con       = get_and_lock_tcp_con_from_sock(tcp_state->con_map, sock);

    if (con->con_state != ESTABLISHED) {
        log_error("TCP connection is not established\n");
        goto err;
    }

    __send_data_pkt(con);
    put_and_unlock_tcp_con(con);
    return 0;

err:
    if (con) put_and_unlock_tcp_con(con);
    return -1;
}
```

Recommended Approach

There are many ways you can approach this project. The only requirements are that you meet the TCP specification detailed above, that your TCP layer interfaces correctly with the rest of the Petnet stack, and that your code builds and works in the specified evaluation environment. We recommend, however, that leverage as much of the included Petnet utility functions as possible. Furthermore, we recommend you take the roughly the following approach.

Important Reading

1. Read Chapter 3 of your textbook (Section 3.5 in particular)
2. Read RFC 793 (Section 3 of the RFC contains a wealth of information related to implementation). The RFC contains a very detailed implementation guide for the TCP protocol. Understanding and following that can be of great help.

Initial Phase (Understanding TCP Data Structures)

3. Fetch, configure, and build Petnet if you have not already done so.
4. Examine the code in udp.c and tcp.c. The existing tcp.c file contains function stubs that you will need to implement. The UDP implementation is functionally complete and demonstrates how to use various parts of the Petnet API. In particular, it will how to correctly receive and transmit

packets as well as manage the transport layer headers. It will also show you how to interact with the various helper functions and data types (e.g. `struct ipv4_addr`).

5. Extend the TCP layer so that it prints arriving packets and socket requests. You should be able to run the stack with this basic implementation, send traffic to it from another terminal using netcat (nc), and see it arrive at your TCP module.
6. Investigate the following files in `petnet/libpetnet/core/`:

   packet.h : API for creating packets and handling protocol headers. Specifically, make sure you understand `create_empty_packet()`.

   ipv4.h: Allows access to the IPv4 protocol header. Make sure you understand how to extract a `struct ipv4_addr` from it, see udp.c for an example.

   tcp.h: Provides `struct raw_tcp_hdr` that will allow you to access the TCP header fields. Again, see udp.c for an example of how to extract and embed these into a packet.

   socket.h: API that allows the transport layer to signal the socket layer when various events occur, as well as to access data in the socket buffers.

Implementing TCP Interactive Data Flow

7. Add code to your TCP module to handle incoming IP packets. Begin by adding code to handle passive opens (i.e. listen and receive SYN). Even without handling socket API calls, you can test this code by using a hard-coded connection representing a passive open.
8. Add code to your TCP module to handle active opens. Again, you do not need to base this on socket API calls. You can hard code the active open for now. Active open corresponds to the `tcp_connect_ipv4()` function. You need to send a SYN to remote side, and successfully detect the reception of SYN-ACK.
9. Add code to your TCP module to handle data transfer. Remember that you do not have to implement congestion control, and only need to implement a Stop-And-Wait protocol for reliability (which provides flow control by limiting the "in-flight" data at any time to a single segment)
10. Add code to your TCP module to handle closes. (You receive a FIN from the remote site or a `tcp_close()` from the socket layer). The closing phase of TCP involves several states. See RFC 793 Section 3, and the reference state transition diagram slide for details on how to implement this part.
11. At this point, your TCP module should be able to carry on conversation with a hard-coded partner. Congratulations! You are finished with the most difficult part!

Implementing TCP Connections

12. At this point investigate `tcp_connection.h` to figure out how to store the state for a connection. Again refer to udp.c and udp_endpoint.c as a simplified example of how this should

work. IMPORTANT: At this point you will need to start handling thread based concurrency, so make sure you have read the "Handling Concurrency" section.

13. Modify your connection creation code to store the state of an individual connection inside a `struct tcp_connection`. NOTE: You will need to modify the definition of the struct in tcp_connection.h to contain the state variables you will need to track.

14. Move the active open implementation into the `tcp_connect_ipv4()` function and create a new `struct tcp_connection` to represent it using `create_ipv4_tcp_con()`

15. Modify your passive open implementation to create a tcp connection object, and call the socket layer to report a TCP connection was accepted with `pet_socket_accepted()`.

16. Tie the connection operations to the socket API. Implement `tcp_close()` (which gets called by `pet_socket_close()`) and signal the socket API using `pet_socket_closed()` when you detect a close on the remote side. This should shut down the connection gracefully, and then remove the connection

17. Add code to transfer data between the TCP and socket layers. Note that the socket layer has an internal data buffer implemented using a ring buffer. The TCP layer should move data into the receive buffer, and pull data from the send buffer. You need to make sure to handle the edge cases of the buffer being either empty or full, and also make sure you correctly signal the socket layer when you pull or push data from the socket buffers.

Testing your tcp_module

18. Now you ought to be able to use your http_client and http_server1 from the project A, simply by prepending "petnet_" to the socket API calls.

Implementing Timeouts

19. At this point you should have a functional TCP layer that works fine as long as there are no lost packets. For the final part of the project, you should implement timeouts into your code. Note that you will need to be careful as you are now adding another source of concurrency that you must deal with. This part will likely be challenging, and you will probably need to leverage full range of accessor methods in tcp_connection.h. Before undertaking this part, you should checkpoint your code up to this point and submit that along with your code that includes timeout handling.

Extra Credit: Flow and congestion control

For extra credit, you may implement the flow control and congestion control parts of TCP as they are described in your textbook and in the other sources. Please note that while this is not much code, it does take considerable effort to get right.

Extra Credit: Bug Bounties

If you think you have found a bug in the Petnet stack implementation, you can submit a bug report at any point and receive anywhere from 1-10 bonus points depending on the severity and whether or not you included an acceptable fix. Bugs consist of problems in the framework leading to incorrect behavior or crashes, deviations from either protocol or socket API specifications that were unintentional or obviously wrong, incorrect comments or documentation, or anything else at the instructor's discretion. Note that to be considered for extra credit the bug report must successfully identify the source of the bug in question. Reports along the lines of "I tried to do this and it crashed" are not acceptable; you need to include the underlying reason for the bug and ideally a fix. Bug reports without a fix are eligible for up to 5 bonus points, while bug reports with a fix are eligible for up to 10 bonus points.

Project Submission

- Your code should be written in C and must compile and run in the Linux environment specified by the course
- You should confine your changes to tcp.c and tcp_connection.h
- You will submit your project in Canvas. You will be expected to provide tcp.c, tcp_connection.h, and a README. If you modify our Makefile for any reason, you should hand that in too. The README should include the names of the project team, a brief specification of work undertaken by each member, and anything specific about your submission that you want to let us know.